Parallelism in the commutation fold

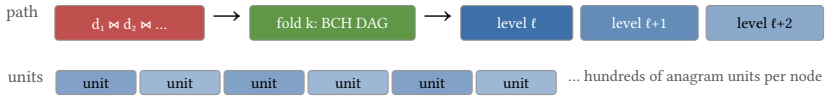
From the algebra of series commutation to the execution schedule of a log-signature computation

signature-rs / lie-rs — 2026-09-02. Audience: readers comfortable with Lyndon bases, Lie series commutation, and log signatures; no familiarity with this codebase assumed. Concepts and methods only.

The three scales of the computation

A log signature of a path is built by folding the displacements of consecutive path points into a running Lie series, one Baker–Campbell–Hausdorff (BCH) concatenation at a time. The whole computation nests three scales, and the honest first answer to "where can threads go?" is different at each scale:

The path (scale 1)	An associative reduction of the per-segment displacements by BCH concatenation: $S = (((d_1 \oplus d_2) \oplus d_3) \oplus \cdots)$. Computed as an **adaptive balanced tournament over chunks** — associativity lets the reduction tree take any shape, and the driver exploits it (§8); the strictly sequential chain remains the single-threaded and small-batch path.	*a data dependency across folds; associativity is the one escape hatch, taken (§8)*
The fold (scale 2)	One concatenation $A \oplus B$ evaluates a fixed **directed acyclic graph** of Lie bracket nodes, compiled once from the BCH series truncated at the working degree. Nodes on one dependency level are independent; levels are dependency-ordered.	*parallel within each level; barriers between levels (§5)*
The commutation (scale 3)	Each node evaluates one series commutation $[X, Y]$ over the Lyndon basis: thousands of independent pair contributions, partitioned into **anagram units** that never share a target word.	*embarrassingly parallel inside a unit-partition (§2–4)*



Nesting: a tournament (or, single-threaded, a chain) of folds; each fold a level-ordered DAG of commutations; each commutation a bag of anagram units. Parallelism exists at every scale — inside a commutation, inside a fold, and across folds through associativity — but each scale's parallelism is bounded by a different structure: unit counts, level widths, and the reduction tree's shape.

The algebra: what one commutation computes

Brackets over the Lyndon basis

A Lie series $A = \sum_u a_u w_u$ is a formal sum over the Lyndon words — the canonical basis of the free Lie algebra on d letters. The commutator of two series distributes over pairs of basis words:

$$[A, B] = \sum_{u<v} (a_u b_v - a_v b_u)[u, v]$$

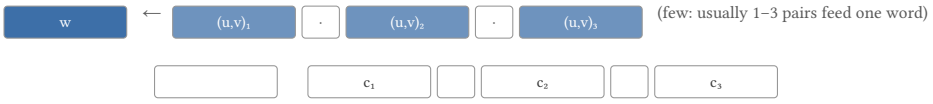
The sum runs over **canonical pairs** $u < v$ of Lyndon words (antisymmetry halves the work immediately). Each bracket $[u, v] = uv - vu$ re-expands in the Lyndon basis; the expansion has two rigid structural symmetries, and they are the entire foundation of the parallel layout:

Degree additivity	$\lvert u\rvert + \lvert v\rvert = t$: every word in the expansion of $[u, v]$ has degree t. Truncating the series at degree m discards all pairs with $\lvert u\rvert + \lvert v\rvert > m$ up front.
Content homogeneity	$[u, v]$ expands only over words with the same letter multiset as uv. Writing $\gamma(w)$ for a word's content vector in $\mathbb{N}^d$: a pair with $\gamma(u) + \gamma(v) = \gamma$ can only ever touch basis words of content γ.

A **feasible decomposition** of a basis word w is a canonical pair (u, v) such that w appears (with nonzero integer coefficient) in the expansion of $[u, v]$, together with that coefficient $c^w_{u,v} \in \mathbb{Q}$. The coefficient of w in $[A, B]$ is then one scalar fan-in over all its feasible decompositions:

$$[A, B]_w = \sum_{(u,v)\to w} c^w_{u,v} \cdot \underbrace{(a_u b_v - a_v b_u)}_{\text{term — 1–2 multiplies}}$$

Each feasible pair contributes its bracket-coefficient times the antisymmetric product of the operand values. All $c^w_{u,v}$ are exact small rationals — precomputable once per basis, before any data arrives.



One word w's coefficient = a short inner product: its (pair, rational) decomposition row dotted with the antisymmetric pair-terms. Both the pair table and the rational rows are **structure constants**: they depend only on the basis, never on the data.

The precomputed structure-constant table

Because every ingredient except the a_u and b_v is data-independent, the entire commutation is driven by a single precomputed table:

Entries	All feasible canonical pairs (i, j), packed as tiny fixed-size records, **degree-blocked**: sorted by (target degree t, then $p = \lvert u\rvert$, $q = t - p$). Degree blocking is what lets a truncated fold skip whole (p, q) regions of the table without testing them.
Decomposition rows	Per entry, the run of (target word, coefficient) pairs it feeds — the words whose expansion contains $[u, v]$. Most rows have length 1–3: almost every pair feeds its own distinguished word (the word uv itself when it is Lyndon) and almost nothing else.
Anagram-unit bounds	Markers (§3) delimiting the groups of entries that can write to the **same** set of target words — the unit is the conflict boundary for parallel writes.

Anagram classes and units: the partition that makes parallelism safe

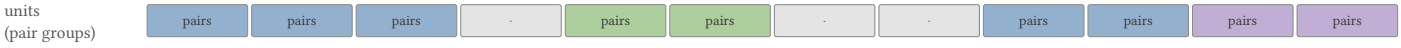
Grouping by content

Group all degree-t basis words by their content vector γ. Each group — an **anagram class** — is the set of words obtainable from one another by permuting letters. Now consider a canonical pair (u, v) with $\gamma(u) + \gamma(v) = \gamma$, $|u| + |v| = t$: by content homogeneity, **everything it produces lands on the single class (t, γ)**.

The kernel groups entries by this pair — a (t, γ)-group is an **anagram unit** — and the payoff is the key structural fact of the whole engine:

> **Two distinct units never write to the same result word.**
>
> Each result word belongs to exactly one anagram class, and each unit feeds exactly one class. Therefore a whole commutation decomposes into units that are **disjoint write regions**: any two of them can be swept concurrently with no locks, no atomics, and no order constraints between them.

A degree-t slice of the result vector, one cell per basis word, coloured by its anagram class: four classes (blue, green, purple and their shades). Words of one class are **contiguous in memory** (the basis is permuted so classes are consecutive — the class-contiguous ordering, §5).



The corresponding units in the entry table: each unit's canonical pairs (one sub-row per unit shown for the blue, green and purple classes; grey cells are units gated off because the current operand supports have no words of those degrees). Every unit scatters only onto words of its own colour above — never anywhere else.

Measured shape of this partition (2 letters, degree ≤ 12): 747 basis words, 212 units, ≈1700 feasible pairs in one commutation; ≈3.5 words and ≈8 pairs per unit. At high degree the classes grow quickly (the class at content $(6, 6)$ holds dozens of words), so units are numerous enough to fill many threads — but sized wildly differently, which matters for scheduling (§5).

Why a unit may never be split

Inside a unit, several canonical pairs feed the **same** result word (the word's decomposition row, §2). The result word accumulates its contributions sequentially — addition of floating-point values is not associative, so **the mathematical definition includes a fixed per-word accumulation order** (the order of the decomposition row). Two workers writing `+=` into the same word would race (lost updates) and would break reproducibility even when they don't (interleaving changes the rounding). Hence:

> **The anagram unit is the atomic scheduling unit.** Packs of parallel work are cut **only** at unit boundaries — a whole unit's entries always stay together, in table order, on one worker. Within a unit the sequential row order is honored by construction; across units no ordering exists at all. This is what makes results **bit-identical to the serial sweep at any thread count** — a property the codebase maintains as an invariant (and guards with cross-thread-count identity tests). The alternative — splitting units and compensating with atomics or privatized partial sums — costs either synchronization on the hot path or extra passes over memory, and an early version of the packer that accidentally cut inside units produced exactly the flaky wrong-results race this boundary exists to prevent.

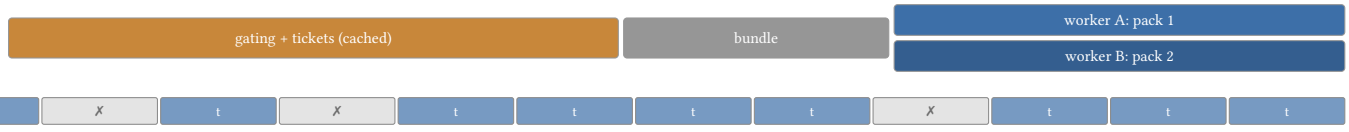
The finer division: write-access classes

The scheduling cell has changed twice; the shipped design (current) is the **unit itself**, made exact. An earlier revision scheduled per **write-access class** — all contributions writing ONE result word, wherever in the table their entries sit — which made the conflict boundary exact but paid for it with a two-phase kernel (a term phase writing per-word staging buffers, then a fan-in phase summing them): the fan-in enumerated each entry once per target **word** instead of once per **unit**, multiplying the sweep's entry visits several-fold, and the staging buffers plus the extra phase barrier dominated small folds. The current engine proves the cheaper division is already exact: **units are unique by (target degree, content), and every table entry that produces a given word lives in that word's one unit — so the units' word sets partition the write slots**. A per-unit task can therefore add its row contributions **directly into the result buffer** (single phase, no staging, no merge), race-free by the same disjoint-write argument as before; and because a word's adds all happen inside its one unit in table-entry order, the per-word accumulation sequence is **bit-identical** to the old per-word design at any thread count. The write-access class survives as the layout and gating concept — the gating walk resolves each unit's exact write set (`unit_words`), scatter sets are that flat list, and packs are cut only at unit boundaries — but the atomic scheduling unit is the unit, and one unit's sweep is one pass: term computed inline, contributions added, done.

One commutation, parallelized end to end

A single evaluation of $[X, Y]$ has four phases; two are parallel, two are serial-but-amortized. The architecture's recurring theme is: **anything whose value depends only on the operand supports (which words are nonzero), not on their values, can be precomputed and cached — supports change rarely, values change every fold.**

① **Gating**	From the two operands' supports, build presence bit-masks, then walk the entry table once to find **active segments**: the degree-runs whose (p, q) could produce anything. Output: per-unit active entry lists.	serial per call, **cached** by support fingerprint
② **Tickets**	For each active segment, resolve **once** the per-entry presence tests (is u present in X? v in Y? both reversed?) and pack the survivors as compact **tickets**: the entry's table index plus two orientation bits.	part of the cached gating — not recomputed per fold
③ **Bundle**	Cut the active units into a handful of **packs** of roughly equal total work, cutting only at unit boundaries.	cheap planning, parallel-safe by §3
④ **Sweep**	Workers claim packs dynamically. Per ticket: compute the antisymmetric term (1–2 multiplies) and add its decomposition-row contributions straight into the result buffer — one phase, direct-add, no staging.	the **only** heavy parallel region



Ticket compaction in one segment: surviving entries (blue) become a dense ticket stream; entries that fail presence (grey, ✗) vanish from the sweep entirely. Measured at 2×12: 31.7% of all entry visits were dead — tickets eliminate both the dead visits and the repeated bit-tests from every fold of the batch, for the price of one pass per support-change.

Two numbers characterize why these phases are arranged this way:

- **The sweep visits hundreds of thousands of entries per call** (measured ≈257,000 tickets per fold at 2×12); the gating/ticket phases are O(table) once per support change. Across a batch of 1000 folds, the per-fold marginal cost of ① and ② is zero.
- **The serial fraction around a single call** (fork/join, task flattening, bundle build) is a fixed few microseconds — negligible against a millisecond sweep at 2×12, but **larger than the entire kernel** at 12×2 (66 entries). This asymmetry returns in §7 as the work-adaptive scheduler.

The sweep itself

Per ticket the arithmetic is minimal: one or two multiplies for the term, then one fused multiply-add per decomposition-row element, added straight into the result (single phase — the sweep is the whole computation, no staging pass). What surrounds it is memory: the entry record, the operand words it indexes, the decomposed coefficients, and the result words. Measured on real hardware, the per-entry time is **uniform** (11 ns/entry at 1 thread, ≈44 cycles) regardless of the pair's degree or row length — which is why balancing packs by entry count balances time (a fancier work metric was implemented, measured slower, and reverted) — and why the sweep parallelizes cleanly until **cache-coherence and streaming bandwidth**, not dependencies, set the limit (per-entry cost rises ≈13% merely because two cores stream the same tables; ≈9× at 32 threads in the worst place — the accumulate traffic of §6).

The fold: a DAG of commutations with level barriers

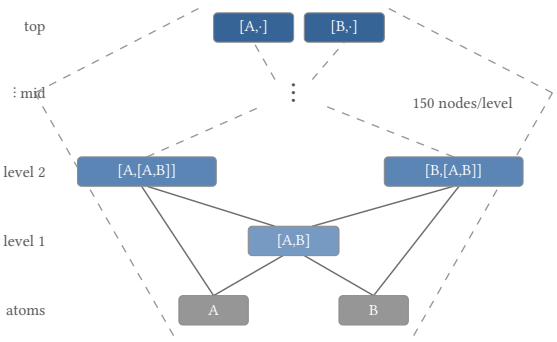
From the BCH series to a DAG

Truncated at degree m, the BCH series $A \oplus B = A + B + \tfrac{1}{2}[A,B] + \tfrac{1}{12}[A,[A,B]] - \tfrac{1}{24}[B,[A,B]] + \cdots$ has a fixed bracket structure — it depends only on m, not on data. The fold compiler turns this bracket forest into a DAG:

Nodes Bracket subterms, **interned** so shared subtrees (the same nested bracket can appear in many BCH terms) are computed once. Each node is one commutation kernel call — and, by content homogeneity, each node writes **exactly one anagram class** of target words, swept as that class's per-unit bundles.

Atoms The two operands: the running accumulator (A) and the folded-in displacement (B).

Levels A node's level is the longest path from the atoms. Nodes of level ℓ read only results of levels $< \ell$, so each level is internally independent — many kernel calls with disjoint result buffers — while level $\ell + 1$ must wait for **all** of level ℓ.



The fold as a DAG — the $[A,B]$-iterate spine of the truncated BCH structure, shown schematically. Solid edges: consumption of an operand (a node's two operands are results of earlier levels and/or the atoms — note how $[A,B]$ is **shared** by every node above it: interning computes it once). The dashed guides suggest the **funnel** shape of the real DAG at working scale (2×12: 11 levels, 600 nodes, 150 nodes at the widest level): thin at the atoms and the deepest levels, wide in the middle. The levels are the longest path from the atoms; width per level is what a parallel schedule can harvest, and the barrier after each full level is what it must pay.

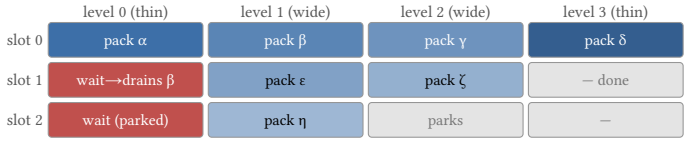
One parallel dispatch for the whole fold: the pack-slot walk

Naively, a fold with L levels pays L separate fork/join dispatches — measured at ≈35% of a small fold's time. The folded engine instead plans **all** levels up front (legal because every node's support is fixed for the fold's duration) and dispatches **once**. Dependency order is enforced not by dispatch boundaries but by a small counter protocol:

Packs per level Each level's (node, unit) tasks are cut into work-balanced packs — again, only at unit boundaries (§3). Because each node is one anagram class and tasks are node-ordered, every pack is **class-contiguous for free**, which keeps a pack's scatter traffic inside a few cache-resident class blocks.

Slots A small number of long-lived worker tasks — never more than workers — each **claims packs off an atomic cursor** per level, executes them, then bumps the level's completion counter.

Gates At each level boundary **every** slot waits for the previous level's counter to reach its pack count (acquire/release ordering). A slot that arrives early is not allowed to sit idle-**and**-blocking: before waiting, it **drains any unclaimed packs of the level itself**. That one rule is the whole liveness proof: a level's remaining work is always either claimed-and-running or claimable by the waiter, so the walk cannot deadlock no matter how the pool is oversubscribed.



The walk across one fold (3 slots, 4 levels; shaded = sweep work, red = waiting). Barriers are implicit in the counters; empty levels or low-work phases are where slots park (and free their workers for co-scheduled work) after draining the cursor. Per-word accumulation order is exactly the serial schedule's: packs never split a unit, and a word's adds all live in its one unit.

The batch: amortizing everything except the serial dependence

A path of 1000 segments is 1000 folds. Running them through the per-fold machinery would re-plan, re-gate, re-gather, and re-allocate 1000 times. The batched driver instead recognizes what is invariant **across** folds and pays it once. The same principle now reaches past the batch: the per-build tables — the structure-constant (feasible-decomposition) table, the BCH series, and the compiled DAG template, all pure functions of (alphabet, degree) — are cached process-wide, so repeated computations on one builder skip straight to folding; and the fold DAG itself is stripped of unreachable (dead) nodes at construction, as an invariant guard on the interned structure.

Support fixed point After the first few folds the accumulator is dense; every node's "which words does it write" list becomes a fixed point. From then on: no per-fold gating, no list rebuilds — the planner's whole output (gates, tickets, packs) is reused verbatim for the remaining folds. Eligibility for batching is exactly this fixed point plus a shared displacement support. The fixed point itself is now **derived once per distinct displacement support and shared process-wide** (a support-keyed cache adopted zero-copy by each worker's DAG via `Arc`-shared, copy-on-write lists), so concurrent leaf groups with the same support — the uniform-workload common case — plan once, not once per group.

Class-space accumulator The accumulator stays in the class-contiguous basis order across the whole batch (gathered once, written back once at the end), removing two full-basis permutations per fold.

Compact node buffers Each node's result lives in a private scratch buffer holding only the degree slices it can touch (degree-blocked, shift-addressed), not the full basis — hundreds of nodes × full-basis buffers would be megabytes and cache-hostile; degree slices cut that 5×.

One stage chain, one walk The whole batch is a single chain of stages — per fold: **gather** the displacement (block-parallel), the L sweep levels, then **accumulate** the terms into the accumulator (block-parallel, simultaneously re-zeroing the node buffers for the next fold) — driven by exactly one pack-slot walk, sharing all cursors and counters.

The batch as one stage chain (5 of L sweep levels shown). Parallel work exists **inside** every stage; between stages, barrier counters; between the last accumulate of fold k and the first sweep of fold $k+1$, a true data dependency — the accumulator. The chain is long and thin by design: the planning and allocation it replaces cost far more than the barrier hops when a fold's sweep work is large, and far less (§7) when it is small.

The accumulate stage deserves one sentence of its own, because it is the batch's second-largest cost: each fold's terms must be added (with BCH weights) into the accumulator, and every node buffer re-zeroed, per fold. It runs block-parallel over class-position ranges — but measured element volume is large (≈575,000 slot-touches per fold at 2×12, ≈85% of the sweep's own multiply count) because node buffers store whole degree slices while only support positions are live. Trimming the walk to live slots and fusing read-and-zero into one pass made this −63% smaller; what remains is **coherence-limited**: per-element cost grows 9× from 4 to 32 threads as the shared accumulator region crosses many cores — the one remaining place where more threads actively hurt inside a stage.

Scheduling to the shape of the work

The one structural number that decides whether parallelism helps is **planned sweep work per fold** — visible exactly, for free, inside the plan. The engine therefore sizes its own parallelism:

	planned entries per fold	chosen slots (32-thread pool)	e2e at 32t vs 1t
12×2 (high-dim, low-degree)	66	1 (serial walk)	2.55 ⟶ 1.25 ms ✓ (was 16.4 ms!)
8×3	700	1	28.6 ⟶ 7.7 ms ✓
3×8	≈74,000	19	338 ⟶ 161 ms ✓
2×12 (low-dim, high-degree)	≈257,000	32	1393 ⟶ 549 ms ✓

Same engine, two shapes, before work-adaptivity (illustrative shares, measured): at 12×2 the fold's 66 entries vanish inside the stage machinery (16t = 6.6× **slower** than 1t); at 2×12 the sweep dominates and 16–32 threads pay. The scheduler's rule — **slots ≈ planned work ÷ a fixed quantum, capped by the pool, floored at 1** — makes each shape run in the regime that suits it, at the cost of nothing when the work is large.

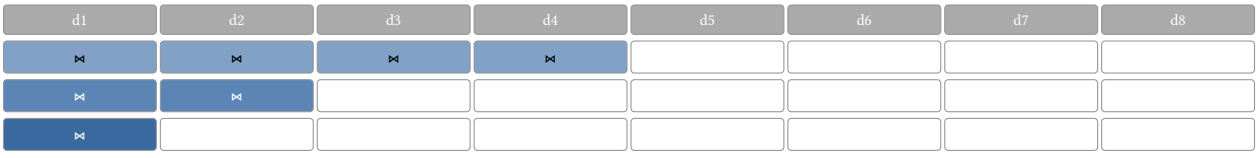
This single rule closed the high-dimension/low-degree pathology: the parallel infrastructure is still there, but a 66-entry fold no longer pays for it.

What threads cannot reach inside a fold — and the associative axis they can

It is worth stating plainly where parallelism **does not** exist, because it is a consequence of the mathematics rather than an implementation gap:

> **Within one concatenation, the accumulator is a data dependency.** Fold $k+1$ of a **sequential chain** evaluates its DAG on the output of fold k; level-0 jobs of fold $k+1$ read the accumulator written by the accumulate stage of fold k. No reordering of stages, no matter how clever, removes that **data** dependency. Everything shown so far — units, levels, packs, slots, batches — lives strictly inside one concatenation; along the path these folds were, in the sequential formulation, replicated identically once per segment.

The one honest escape hatch already exists in the algebra — and the engine takes it: BCH concatenation is **associative**. The path's log signature is an associative reduction of the segment displacements, so the reduction tree may have any shape — including a balanced tournament:

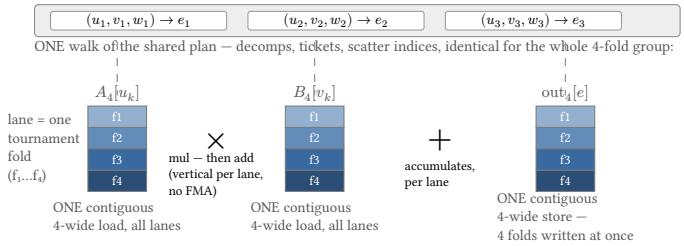


A tournament reduction over 8 segments: $\log_2 n$ sequential rounds, every pair in a round independent. Rounds start **sparse** (degree-1 displacements: cheap, support-gated folds) and only the last $\sim 2\log_2 n$ concatenations along the critical path pay full dense-fold cost. A 1000-segment path: from 1000 serial dense folds to 10 of dependency depth, up to 500 concurrent in round 1.

The engine now performs exactly this reduction, adaptively and with no new API surface: when a batch holds at least 16 displacements on a multi-thread pool, the batch driver reduces a **balanced tournament over contiguous chunks** instead of the serial chain. Leaves are not cold per-fold dispatches — each leaf folds its chunk through the **steady-chunk leaf engine**: one reachable-support plan per chunk (derived once, then shared process-wide across groups with the same displacement support), then the chunk's whole fold sequence as ONE batched dispatch — no per-fold warm-up walks anywhere. Adjacent groups of two-to-four leaves with support-uniform displacements collapse further into ONE SoA cohort dispatch (up to 4 folds per plan walk from the very first fold). The per-leaf warm-up duplication that an earlier leaf design paid — measured up to 1.27× on 2×12 — is gone with the warm-ups themselves. The tree shape is a pure function of the displacement count (at most 32 leaves), so results are **bit-stable at any pool size**; exact (rational)

coefficients remain bit-identical to the sequential driver, while f64 reassociation shifts rounding only in the last ulps (measured $\approx 10^{-13}$ abs on $\sim 10^{2}$ magnitudes — accepted, and pinned by tolerance tests against the sequential oracle plus exactness oracles for rationals). The §7 discipline applies at the tree level too: leaves and merges run through the same work-adaptive machinery, so a thin regime's early rounds do not pay for scheduling. Measured end-to-end (full grid in the regime map below): 12×2 1.84⟶0.50 ms @8t (6.7 × at 10000 segments), 8×3 7.94⟶2.08 ms @32t, 3×8 154⟶92 ms @32t, and 2×12 — the regime that **needs** the intra-fold machinery most — 525⟶390 ms @16t (2.0 × at 10000 segments): 128 batched leaves plus 127 merges replace 1000 dense folds. Single-threaded runs are unaffected: the driver keeps the sequential chain there, so the reformulation is upside-only on every measured grid.

The tournament's folds then compose with a second multiplier, **cohort execution**. A round's folds all walk the same data-independent plan, so the engine executes four of them as one SIMD walk over lane-interleaved buffers:



Cohort execution of one 4-fold tournament group. Coefficient buffers are interleaved per index (`[idx*4+lane]`): every index access of the shared plan walk touches all four lanes in ONE contiguous 4-wide load/store — no gather/scatter instructions anywhere; arithmetic is vertical per lane (mul, then add — deliberately **no** FMA, proven worthless in this load/scatter-bound phase). Indices, gating tickets, and control flow are shared; only coefficient values differ per lane. Each lane therefore replicates the scalar fold's operation order exactly, so a cohort run is **0-ulp bit-identical** to the scalar tournament over the same reduction tree. The kernel is capability-gated: f64/f32 (including `NotNan`-wrapped) via TypeId + runtime AVX2 check, with an environment kill switch; every other coefficient type keeps the scalar kernel bit-for-bit. Engagement was **measured-out** when the engine shipped (a wide-pool gate disabled cohorts above 16 threads for light folds); after the single-phase rewrite removed the two-phase machinery the gate existed to protect, that gate was deleted — the counterfactual matrix found no regime, at any pool width, where gating the cohort off wins (8×3@32t, the cell that motivated it, went 1.54⟶1.09 ms with the gate removed). A/B at introduction: −32% 2×12@4t, −37% 3×8@4t, −29% 8×3@4t (the old 32-thread time on 4 threads), −20% 2×12@8t; neutral elsewhere. Absolute bests unchanged — cohorts buy that 4–8 threads now reach 1.5× of the 16–32-thread times.

Regime map: what was measured

grid	stock, best config	today, best config	gain	why
2×12	1393 ms @32t	390 ms @16t	3.6×	Every intra-fold layer engaged — plus the tournament: 128 batched leaves + 127 merges replace 1000 dense folds (2.0 × alone at 10000 segments). *(low-d, high-m)*
3×8	331 ms @8t	92 ms @32t	3.6×	Same; tournament gain grows with pool (1.7 × @32t).
12×2	2.55 ms @1t (16t: 16.7 ms!)	0.50 ms @8t	5.1×, pathology gone — and threads come back	66 entries/fold: adaptivity runs the fold serial — the tournament puts map lanes on the fold AXIS (6.7 × vs stock at 10000 segments). *(high-d, low-m)*
8×3	8.02 ms @2t (32t: 28.6 ms)	2.08 ms @32t	3.9×	700 entries/fold: formerly on the serial/parallel boundary — same mechanism as 12×2.

End-to-end log signatures of 1000-segment random paths, criterion medians, 32-core machine. "Stock" = the engine before the parallelism work documented in the companion notes. Single-threaded runs keep the sequential driver (bit-identical there); parallel paths reassociate floating-point folds — exact types unchanged, ulps pinned by oracle tests.

Update, 2026-09-05 (current head). The table above is kept as the campaign's record. Since it was measured, the kernel's scheduling cell became the anagram unit with a single-phase direct-add sweep (§3.3), per-build tables and derived plans became process-wide caches, and cohort engagement became purely capability-based. Fresh 1000-segment e2e medians on the same machine, current head: 8×3 **1.09 ms** @32t (was 2.08 in the table — the last gate removal), 2×12 **216 ms** @32t, 3×8 **48 ms** @32t, 4×6 **20.0 ms** @4t, 2×8 **3.99 ms** @32t. The regime shape is unchanged: low-degree grids stay dispatch-bound and high-degree grids stay bandwidth-bound; the constants moved.

Summary. Parallelism in this engine is **structural**, not opportunistic: the algebra partitions work into write regions that never share a word — anagram units at both storage and scheduling granularity (their word sets partition the write slots, which is what makes the single-phase direct-add sweep race-free and bit-exact); the fold DAG partitions those pieces into dependency levels; the batch amortizes all support-invariant planning — down to sharing derived plans and steady-state lists process-wide across groups with the same displacement support; and the scheduler spends threads only where measured work justifies the barriers. The fold chain along the path is a data dependency of the BCH recursion itself — and the tree reduction of §8, now performed adaptively by the batch driver, is the only mechanism mathematics offers for crossing it.